

DOKUZ EYLUL UNIVERSITY
ENGINEERING FACULTY
DEPARTMENT OF ELECTRICAL AND ELECTRONICS
ENGINEERING

EED 1005
INTRODUCTION TO PROGRAMMING
PROGRESS REPORT
PROJECT

BINARY SUDOKU

by

Ömer Ege NİŞLİ

İsmail ÖZTÜRK

Mehmetcan KIRCA

Muhammet Emre AY

Zeynep SEYHUN

Halil Ilgaz KESKİN

Yiğithan YILDIRIM

Lecturer

Ögr. Gör. Dr. Özlem ÖZTÜRK

İZMİR

17.12.2024

CHAPTER ONE

PROGRESS DESCRIPTION

During the first week of the project, we made our studies about the topic of the project, then we divided the project to little parts and created the flowchart. After that we tried to figure out the ways to complete our parts.

During the second week of the project, we created the board of the game and the variables required for it. Then we created the main part of the game code.

During the third week of the project, we fixed the errors that we encountered during the process and modified the part that places the pieces on the board with the coordinates given by the player. After that we wrote a progress report.

The purpose of the project was to understand the topics of c programming that we learn in our online lessons and experience to write a game with them.

Throughout this time we didn't conduct any online research for the project. We used the Power Point slays and the documents that given to us as a source in the sakai web page.

CHAPTER TWO

TASK SUMMARY

2.1 Completed Tasks

Halil Ilgaz KESKİN and Yiğithan YILDIRIM: We created the game board.

Muhammet Emre AY: I did the part which creates the random pieces with Zeynep SEYHUN.

Ömer Ege NİŞLİ: I did the parts that places the random piece to the board by the user input, controlling the coordinates given by user if there is any overlapping pieces and the control of the full rows, collums and 3*3 squares then popping them.

Mehmetcan KIRCA: I did the part that controls the board to see if there are any empty places to fit the next piece if not it ends the game. And I combined the functions and completed the main code with the help of İsmail ÖZTÜRK .

İsmail ÖZTÜRK: I combined the functions and completed the main code with the help of Mehmetcan KIRCA, did the part which calculates the score and created the flowchart.

Zeynep SEYHUN: I did the part which creates the random pieces with Muhammet Emre AY and wrote the progress report.

2.2 Incomplete Tasks: Reasons and Explanations

Halil Ilgaz KESKİN and Yiğithan YILDIRIM: All tasks were completed.

Muhammet Emre AY: All tasks were completed.

Ömer Ege NİŞLİ: All tasks were completed.

Mehmetcan KIRCA: All tasks were completed.

İsmail ÖZTÜRK: I wrote the score calculating function but my function doesn't work properly and calculates the score wrong .

Zeynep SEYHUN: All tasks were completed.

2.3 Additional Problems

Halil Ilgaz KESKİN and Yiğithan YILDIRIM : The board was looking too complicated when we placed the pieces, that's why we improved it and made it look more organized.

İsmail ÖZTÜRK and Mehmetcan KIRCA: The game wasn't ending when we created the main code then we fixed the flaw. Now the game ends without a problem.

CHAPTER THREE

EXPLANATION OF THE ALGORITHMS

3.1 Screenshots

Placing the Pieces:

```
Welcome to Binary Sudoku!
  A B C D E F G H I
+---+---+---+
1 | . . . | 0 1 0 | . . 0 |
2 | 0 0 0 | . . 0 | . . 0 |
3 | 0 . . | 1 . . | . 1 0 |
+---+---+---+
4 | 0 1 1 | . . . | . 1 . |
5 | 0 . 1 | 1 . . | . 1 . |
6 | 1 . . | 0 . . | . . 0 |
+---+---+---+
7 | . . . | 0 1 . | . . . |
8 | . . 0 | . . . | 1 1 . |
9 | . 1 . | . . . | . . . |
+---+---+---+
Score: 0

New Piece:
1
1
1
Enter location (ex A1): A7
```

```
Enter location (ex A1): A7
  A B C D E F G H I
+---+---+---+
1 | . . . | 0 1 0 | . . 0 |
2 | 0 0 0 | . . 0 | . . 0 |
3 | 0 . . | 1 . . | . 1 0 |
+---+---+---+
4 | 0 1 1 | . . . | . 1 . |
5 | 0 . 1 | 1 . . | . 1 . |
6 | 1 . . | 0 . . | . . 0 |
+---+---+---+
7 | 1 . . | 0 1 . | . . . |
8 | 1 . 0 | . . . | 1 1 . |
9 | 1 1 . | . . . | . . . |
+---+---+---+
Score: 0

New Piece:
1 0
1 1
Enter location (ex A1): G1
```

```
Enter location (ex A1): G1
  A B C D E F G H I
+---+---+---+
1 | . . . | 0 1 0 | 1 0 0 |
2 | 0 0 0 | . . 0 | 1 1 0 |
3 | 0 . . | 1 . . | . 1 0 |
+---+---+---+
4 | 0 1 1 | . . . | . 1 . |
5 | 0 . 1 | 1 . . | . 1 . |
6 | 1 . . | 0 . . | . . 0 |
+---+---+---+
7 | 1 . . | 0 1 . | . . . |
8 | 1 . 0 | . . . | 1 1 . |
9 | 1 1 . | . . . | . . . |
+---+---+---+
Score: 0

New Piece:
1 0
1 1
Enter location (ex A1): G6
```

```
Enter location (ex A1): G6
  A B C D E F G H I
+---+---+---+
1 | . . . | 0 1 0 | 1 0 0 |
2 | 0 0 0 | . . 0 | 1 1 0 |
3 | 0 . . | 1 . . | . 1 0 |
+---+---+---+
4 | 0 1 1 | . . . | . 1 . |
5 | 0 . 1 | 1 . . | . 1 . |
6 | 1 . . | 0 . . | 1 0 0 |
+---+---+---+
7 | 1 . . | 0 1 . | 1 1 . |
8 | 1 . 0 | . . . | 1 1 . |
9 | 1 1 . | . . . | . . . |
+---+---+---+
Score: 0

New Piece:
1 1
Enter location (ex A1): H9
```

```

Enter location (ex A1): H9
  A B C D E F G H I
+---+---+---+---+
1 | . . . | 0 1 0 | 1 . 0 |
2 | 0 0 0 | . . 0 | 1 . 0 |
3 | 0 . . | 1 . . | . . 0 |
+---+---+---+---+
4 | 0 1 1 | . . . | . . . |
5 | 0 . 1 | 1 . . | . . . |
6 | 1 . . | 0 . . | 1 . 0 |
+---+---+---+---+
7 | 1 . . | 0 1 . | 1 . . |
8 | 1 . 0 | . . . | 1 . . |
9 | 1 1 . | . . . | . . 1 |
+---+---+---+---+
Score: 511

New Piece:
1
Enter location (ex A1):

```

Finishing The Game:

```

Enter location (ex A1): A1
  A B C D E F G H I
+---+---+---+---+
1 | 1 1 1 | 0 1 0 | 1 . 0 |
2 | . 0 0 | . . 0 | 1 . 0 |
3 | 1 1 . | 1 1 1 | 1 0 0 |
+---+---+---+---+
4 | . 1 1 | . . . | 1 1 . |
5 | 0 1 1 | 1 1 . | 1 1 1 |
6 | 1 1 . | 0 . . | 1 . 0 |
+---+---+---+---+
7 | 1 1 1 | 0 1 . | 1 1 0 |
8 | 1 . 0 | . . . | 1 1 1 |
9 | . 1 . | . 1 . | . . 1 |
+---+---+---+---+
Score: 1022

New Piece:
0 1
1 1
Enter location (ex A1): E8
Invalid move, try again.

```

```

Enter location (ex A1): D7
  A B C D E F G H I
+---+---+---+---+
1 | 1 1 1 | 0 1 0 | 1 . 0 |
2 | . 0 0 | . . 0 | 1 . 0 |
3 | 1 1 . | 1 1 1 | 1 0 0 |
+---+---+---+---+
4 | . 1 1 | . 1 . | 1 1 . |
5 | 0 1 1 | 1 1 . | 1 1 1 |
6 | 1 1 . | 0 . 1 | 1 . 0 |
+---+---+---+---+
7 | . . . | 1 1 . | . . . |
8 | 1 . 0 | 1 0 . | 1 1 1 |
9 | . 1 . | . 1 . | . . 1 |
+---+---+---+---+
Score: 1533

New Piece:
1
1
1
Enter location (ex A1): F7
Game over! Final Score: 1533

```

3.2 Functions

The random board initializing function given by:

```
void initialize_random_board(void) {
    int filled_cells = 0; // Number of cells to fill with 1's and 0's

    for (int i = 0; i < SIZE; i++) {
        for (int j = 0; j < SIZE; j++) {
            if (filled_cells < ismail) {
                if (rand() % 2 == 0) { // 50% probability we place 1 or 0
                    board[i][j] = rand() % 2; // 1 or 0 placed randomly
                    filled_cells++; // A cell was placed
                }
            }
            else {
                board[i][j] = -1; // The cell is left blank (-1 represents an empty cell)
            }
        }
        else {
            board[i][j] = -1; // After 27 cells are filled, it is left blank
        }
    }
}
```

The board printing function given by:

```
void printBoard(void) {
    printf("    A  B  C  D  E  F  G  H  I");
    printf("\n  + - - - - + - - - - + - - - - +");
    for (int i = 0; i < SIZE; i++) {
        printf("\n%d |", i + 1);
        for (int j = 0; j < SIZE; j++) {
            if (j == 2 || j == 5 || j == 8) {
                if (board[i][j] == -1) {
                    printf(" . "); // We fill the empty cells with dots
                }
                else {
                    printf(" %d |", board[i][j]);
                }
            }
            else {
                if (board[i][j] == -1) {
                    printf(" . ");
                }
                else {
                    printf(" %d  ", board[i][j]);
                }
            }
        }
        if (i == 0 || i == 1 || i == 3 || i == 4 || i == 6 || i == 7) {
            printf("\n  |          |          |          |");
        }
        if ((i + 1) % 3 == 0) {
            printf("\n  + - - - - + - - - - + - - - - +");
        }
    }
    printf("\nScore: %d\n", score);
}
```

The creating random pieces function given by:

```
void generate_piece(int piece[4][4], int* width, int* height) {
    int num = rand() % 10;
    *width = piece_widths[num];
    *height = piece_heights[num];

    for (int i = 0; i < *height; i++) {
        for (int j = 0; j < *width; j++) {
            piece[i][j] = pieces[num][i][j];
        }
    }
}
```

The function places the pieces given by:

```
int place_piece(int piece[4][4], int width, int height, int row,
int col) {
    for (int i = 0; i < height; i++) {
        for (int j = 0; j < width; j++) {
            if (piece[i][j] != 0) { // check 1010
                if (row + i >= SIZE || col + j >= SIZE || board[row
+ i][col + j] != -1) {
                    return 0; // cant_place
                }
            }
        }
    }

    //placepiece
    for (int i = 0; i < height; i++) {
        for (int j = 0; j < width; j++) {
            if (piece[i][j] != 0 || piece[i][j] == 0) { //write
                1010
                board[row + i][col + j] = piece[i][j];
            }
        }
    }
    return 1; // success
}
```

The function checks if the place empty to place the piece given by:

```
int valid_invalid(int piece[4][4], int width, int height, int row,
int col) {
    for (int i = 0; i < height; i++) {
        for (int j = 0; j < width; j++) {
            if (piece[i][j] != 0) {
                if (row + i >= SIZE || col + j >= SIZE || board[row
+ i][col + j] != -1) {
                    return 0; // unplaceable
                }
            }
        }
    }
    return 1; // PLACABLE
}
```

The function that checks if a row, a column or a square is full is given by:

```
void check_clear() {
    int multiplier = 0;
    int tempScore = 0;

    // ROW check (binary calculation from right to left)
    for (int i = 0; i < SIZE; i++) {
        int full = 1;
        int binaryValue = 0;
        for (int j = SIZE - 1; j >= 0; j--) {
            if (board[i][j] == -1) {
                full = 0;
                break;
            }
            binaryValue |= (1 << (SIZE - 1 - j)); // build the
                binary number from right to left
        }
        if (full) { // Row is full
            tempScore += binaryValue; // Add the binary value as
                score
            multiplier++;
            for (int j = 0; j < SIZE; j++) {
                board[i][j] = -1;
            }
        }
    }

    // COL check (binary calculation from top to bottom)
    for (int j = 0; j < SIZE; j++) {
        int full = 1;
        int binaryValue = 0;
        for (int i = 0; i < SIZE; i++) {
            if (board[i][j] == -1) {
                full = 0;
                break;
            }
            binaryValue |= (1 << (SIZE - 1 - i)); // build the
                binary number from top to bottom
        }
        if (full) { // Column is full
            tempScore += binaryValue; // Add the binary value as
                score
            multiplier++;
            for (int i = 0; i < SIZE; i++) {
                board[i][j] = -1;
            }
        }
    }

    // 3x3 blocks check (binary calculation from right to left, row
        by row)
    for (int blockRow = 0; blockRow < 3; blockRow++) {
        for (int blockCol = 0; blockCol < 3; blockCol++) {
            int full = 1;
            int binaryValue = 0;
            for (int i = blockRow * 3; i < (blockRow + 1) * 3; i++)
            {
                for (int j = blockCol * 3; j < (blockCol + 1) * 3;
                    j++) {
                    if (board[i][j] == -1) {
                        full = 0;
                        break;
                    }
                }
                binaryValue |= (1 << (SIZE - 1 - j)); // build
                    the binary number from right to left for
                    each row in the block
            }
            if (!full) break;
        }
    }
}
```



```

    }
    if (full) { // Block is full
        tempScore += binaryValue; // Add the binary value
        as score
        multiplier++;
        for (int i = blockRow * 3; i < (blockRow + 1) * 3;
            i++) {
            for (int j = blockCol * 3; j < (blockCol + 1) *
                3; j++) {
                board[i][j] = -1;
            }
        }
    }
}
}
}

```

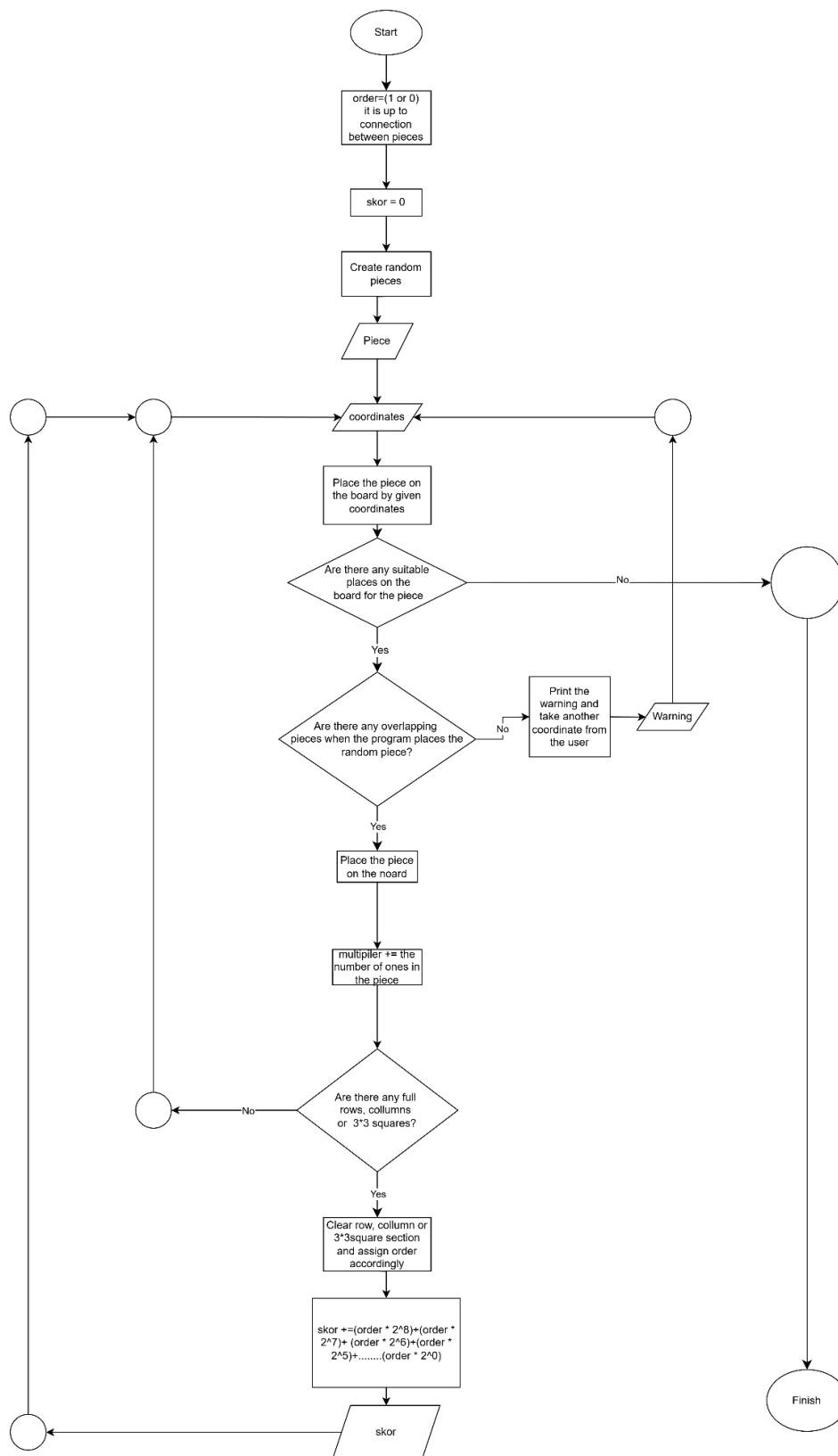
The function that decides to finish the game given by:

```

// CheckEmpty
int game_over(int piece[4][4], int width, int height) {
    for (int i = 0; i < SIZE; i++) {
        for (int j = 0; j < SIZE; j++) {
            if (valid_invalid(piece, width, height, i, j)) {
                return 0;
            }
        }
    }
    return 1; //no placement
}

```

Algorithms and Solution Strategies



CHAPTER FOUR

PROBLEMS ENCOUNTERED

Ömer Ege NİŞLİ: When we tried to place the pieces they were overlapping that's why I changed the code and made it print a warning message when user tries to enter a coordinate that overlaps the pieces. Second thing I improved was popping the rows, columns and 3*3 squares cause that function wasn't working properly. Third thing I improved was "j" column; when the user entered a coordinate included "j" column the code didn't printed the piece on the screen that's why I solved the problem.

İsmail ÖZTÜRK: We encountered a problem when the program calculated the game score. It was giving irrelevant score values when a row, column or 3*3 square was filled by the user. We still couldn't solve that problem.

CHAPTER FIVE

CONCLUSION

Briefly during this three weeks with this project we created the game with 0 and 1 numbers randomly disturbed into the board. In our code the user can place the pieces on the board without any problem. When the user places the random piece to a suitable spot and if a row, a column or a 3*3 square is filled by the user's move , the code can pop those parts and the user can place other pieces to that place. If the last piece placed by the user can't pop a row, a column or a square and the next piece given by the program can't fit into any kind of empty place the program ends itself.

During the making of the project we encountered a lot of errors. But we managed to overcome those errors. Now our program nearly works without a flaw. Only missing part is calculating the game score and we believe we can fix this problem till the final report.

REFERENCES

- Deitel , Harvey ; Deitel , Paul. C How to Program (1992) , Pearson.